

CODEPROMPTZIP: Code-Specific Prompt Compression for Bugs2Fix

Ansh Gupta (IMT2023540) Mayank Tanwar (IMT2023012) Prakrititz Borah (IMT2023547)
Kunal Jindal (IMT2023049)

1. Problem Definition / Scope

Modern LLM-based code repair pipelines use **Retrieval-Augmented Generation (RAG)**: before asking the model to fix a bug, similar buggy–fixed code pairs are retrieved from a knowledge base and placed into the prompt as demonstrations. While effective, this creates two critical bottlenecks:

- **Context window limits:** retrieved examples containing full buggy and fixed methods consume thousands of tokens, often exceeding the LLM’s context window.
- **High inference cost:** proprietary APIs charge per token, making bloated prompts prohibitively expensive.

Existing prompt compression methods (LLMLingua, RECOMP, LLMLingua-2) were designed for natural language and fail on code because they do not consider the **syntactic role** of tokens. An entropy-based approach might discard a method invocation like `.getInstance()` because its sub-tokens are predictable, even though it is semantically critical for understanding a bug.

CODEPROMPTZIP addresses this gap by introducing a **code-specific prompt compression framework** that leverages program analysis to understand which tokens are safe to remove for a given downstream task. This project implements and evaluates the complete CODEPROMPTZIP pipeline on the **Bugs2Fix** task, where a buggy Java method is given as input and the model must output the corrected version.

2. Planned Approach with Justification

2.1 Why CODEPROMPTZIP?

We chose to implement CODEPROMPTZIP over alternative compression methods for three reasons:

1. **Task-specific compression:** Unlike generic methods, CODEPROMPTZIP derives a token removal priority ranking specific to each downstream task (Bugs2Fix), ensuring that only tokens irrelevant to the repair signal are removed.
2. **Controllable compression ratio:** The framework allows specifying an exact compression ratio τ_{code} at inference time via special tokens, unlike summary-based methods that produce variable-length summaries.
3. **Copy mechanism:** The pointer-generator architecture ensures the compressed output is strictly a subset of the input tokens, no hallucinated code, which is critical for code tasks.

2.2 Framework Architecture

The framework operates in two phases.

Training Phase



Figure 1: Training pipeline: from knowledge base to fine-tuned compressor.

Inference Phase

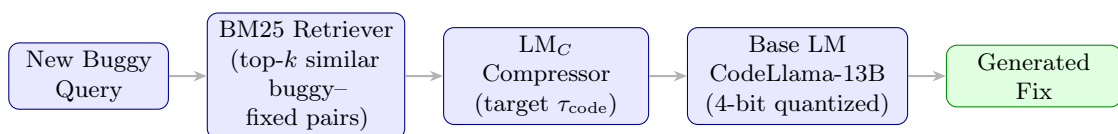


Figure 2: Inference pipeline. LM_C and the base LM are fully decoupled: compressed output is plain text.

The compressor (LM_C) and the base LM are **fully decoupled**: the compressed output is plain text, requiring no gradient sharing or special embeddings between models.

2.3 Key Technical Components

2.3.1 Type-Aware Priority Ranking

Using JavaParser to construct Abstract Syntax Trees (ASTs), every token in a code snippet is categorized into one of five types:

Table 1: Token type definitions and examples

Token Type	Definition	Example
Symbol	Operators, delimiters, punctuation	=, {, ;
Signature	Method declaration and parameters	public static void init(...)
Invocation	Function/method calls	Calendar.getInstance()
Identifier	Variable names, class names	VAR_1, TYPE_1
Structure	Control-flow keywords	if, for, return

The removal priority is computed as:

$$\text{Priority}(T) = \frac{\tau_{\text{code}/T}}{d_T} \quad (1)$$

where $\tau_{\text{code}/T}$ is the compression ratio achieved by removing all tokens of type T , and d_T is the resulting CodeBLEU degradation. Higher priority means a token type saves many tokens with minimal quality loss.

Bugs2Fix Priority Ranking (our configuration):

Identifier > Invocation > Structure > Symbol > Signature

(remove first $\rightarrow$ remove last)

This ranking was adopted directly from the ablation study performed by the original authors, as they demonstrated it to be universally effective across different base LLMs for the Bugs2Fix task. In their ablation study, the authors systematically removed tokens of each specific type in isolation and measured the resulting percentage degradation in CodeBLEU to empirically determine which token types were safest to discard (highest priority) and which were most critical (lowest priority).

2.3.2 Dataset Construction via Priority-Driven Greedy Algorithm

For each parsable code example in the training set, the greedy algorithm:

1. Assigns each token a removal priority based on its type and within-type frequency.
2. Computes $L_{\text{rm}} = \lfloor \tau_{\text{code}} \times L \rfloor$: the number of tokens to remove.
3. Iteratively removes the highest-priority tokens until the budget is exhausted.
4. Returns the compressed code as the training target.

This is repeated for **9 compression ratios** ($\tau_{\text{code}} \in \{0.1, 0.2, \dots, 0.9\}$), producing the following dataset:

Table 2: Dataset composition (source: Microsoft CodeXGLUE Bugs2Fix)

Component	Count
Raw Bugs2Fix pairs	52,364
Compression ratios per example	9
Total compression triples	$52,364 \times 9 = \mathbf{471,276}$
Training split (80%)	377,020
Validation split (10%)	47,128
Test split (10%)	47,128

The split is performed at the code-example level (not at the triple level) to prevent data leakage, all 9 compressed versions of the same code example are assigned to the same split.

Compression Progression Example

To illustrate the priority-driven greedy removal algorithm in action, consider how the following buggy method degrades as the compression ratio (τ_{code}) increases.

```
public static TYPE_1 init(java.lang.String name, java.util.Date date) {
    TYPE_1 VAR_1 = new TYPE_1();
    VAR_1.METHOD_1(name);
    java.util.Calendar VAR_2 = java.util.Calendar.getInstance();
    VAR_2.METHOD_2(date);
    VAR_1.METHOD_3(VAR_2);
    return VAR_1;
}
```

Listing 1: Original ($\tau_{\text{code}} = 0.0$) — 0% tokens removed

```
public static TYPE_1 init(java.lang.String name, java.util.Date date) {
    TYPE_1 = new TYPE_1();
    .METHOD_1(name);
    java.util.Calendar = java.util.Calendar.getInstance();
    .METHOD_2(date);
    .METHOD_3(VAR_2);
    return ;
}
```

Listing 2: Light Compression ($\tau_{\text{code}} = 0.3$) — 30% target. Redundant high-frequency Identifier tokens (VAR_1, VAR_2) begin dropping first.

```
public static TYPE_1 init(java.lang.String name, java.util.Date date) {
    = new TYPE_1();
    ;
    java.util.Calendar = java.util.Calendar;
    .METHOD_2(date);
    .METHOD_3(VAR_2);
    return ;
}
```

Listing 3: Moderate Compression ($\tau_{\text{code}} = 0.5$) — 50% target. Almost all Identifier tokens drop. Core structural context remains intact.

2.3.3 Compressor Architecture: Copy-Enhanced CodeT5

The compressor is built on **CodeT5-base** (Salesforce, 220M parameters), an encoder-decoder transformer pre-trained on 8.35M code functions across 8 programming languages.

Modification 1 — Extended Vocabulary: Special tokens are added to signal the task and compression ratio:

```
<BUGS2FIX> <Ratio> 0.3 </Ratio> <Compress> {code} </Compress>
```

This enables the same model to be steered to different compression levels at inference time.

Modification 2 — Copy Mechanism (Pointer-Generator): The critical architectural addition. At each decoding step t :

1. The decoder produces cross-attention weights $\mathbf{a}^t$ over source positions.
2. A context vector is computed: $\mathbf{h}_t^* = \sum_i a_i^t \cdot \mathbf{h}_i$
3. A generation gate is computed: $p_{\text{gen}} = \sigma(W_{\text{gen}} \cdot [\mathbf{h}_t^*, \mathbf{s}_t] + b_{\text{gen}})$
4. The copy distribution sums attention over matching source tokens: $P_{\text{copy}}(y) = \sum_{i: x_i=y} a_i^t$
5. The final distribution blends vocabulary and copy:

$$P(y) = p_{\text{gen}} \cdot P_{\text{vocab}}(y) + (1 - p_{\text{gen}}) \cdot P_{\text{copy}}(y) \quad (2)$$

This ensures the compressed output is drawn strictly from the input tokens, preventing hallucinated code. The **CopyModule** is the only component initialized from scratch; all other weights are loaded from the pre-trained CodeT5-base checkpoint.

3. Extension and Critique of the Original Work

What We Replicated

- The complete CODEPROMPTZIP pipeline: type-aware priority ranking, greedy dataset construction, copy-enhanced CodeT5 compressor, and BM25-based RAG evaluation.
- The compression ratio sweep experiment, confirming the non-trivial relationship between compression and downstream quality.

Key Differences from the Original Paper

Table 3: Comparison with the original paper

Aspect	Original Paper	Our Implementation
Base LM	GPT-3.5-turbo and Gemini-1.0	CodeLlama-13B (4-bit quantized, local)
CodeT5 variant	CodeT5-large (770M)	CodeT5-base (220M)
Training epochs	10	3 (~ 100 hrs on RTX 4060)
Training data	48,903 parsable $\times 9 = 440K$	52,364 total $\times 9 = 471K$
Eval. samples	Full test set (6,545)	100 samples (resource constraint)

Critique

1. **Base LM sensitivity:** The original paper does not deeply explore how the choice of base LM affects the utility of compressed demonstrations. Our results show that CodeLlama-13B under 4-bit quantization behaves very differently from GPT-3.5-turbo specifically, it shows much less benefit from RAG demonstrations and degrades sharply with multi-shot compressed prompts.
2. **Compression at low τ :** The $\tau = 0.1$ expansion behavior (producing more tokens than the input) is a known but under-discussed limitation. The copy mechanism, while powerful, does not enforce a hard length constraint.
3. **Evaluation scale:** The paper’s results on the full test set (6,545 samples) are more statistically robust than our 100-sample evaluation, which may not capture the full distribution of bug types and complexities.

4. Implementation Details

4.1 Hardware and Environment

Table 4: Hardware and environment specifications

Component	Specification
GPU	NVIDIA GeForce RTX 4060 (8 GB VRAM)
System Environment	Windows (local, RTX 4060)
Compressor for Finetuning	CodeT5-base(220M)
Base LM for evaluation	CodeLlama-13B-Instruct (Q4_K_M quantized, GGUF)
Evaluator	CodeBLEU
Python version	3.12
Framework	PyTorch + HuggingFace Transformers

4.2 Training Configuration

Table 5: Training hyperparameters

Hyperparameter	Value
Base model	Salesforce/codet5-base (220M params)
Micro batch size	2
Gradient accumulation steps	8
Effective batch size	16
Learning rate	5×10^{-5}
Number of epochs	3
Total training steps	$\sim 70,500$ (23,500 steps/epoch $\times 3$)
Total training time	~ 100 hours

These hyperparameters were set to fit the model within 8 GB VRAM. The effective batch size of 16 was achieved via gradient accumulation (micro-batch 2×8 accumulation steps).

4.3 Compressor Fine-Tuning Results

The CodeT5-base model was fine-tuned for 3 full epochs on the 377,020 training samples. Key observations from the training curves:

- **Epoch 1:** The loss dropped dramatically from 3.67 (step 100) to 0.0284 (step 23,000), reflecting the model rapidly learning to compress code using the rich pre-trained representations from CodeT5.
- **Epoch 2:** The validation loss improved from 0.0284 to 0.0145, a 49% reduction confirming that a second epoch over the large dataset was valuable.
- **Epoch 3:** Continued to reduce, with the final best model achieving val loss = 0.0145 at step 47,000. The loss stabilized in the 0.017–0.018 range, indicating convergence.

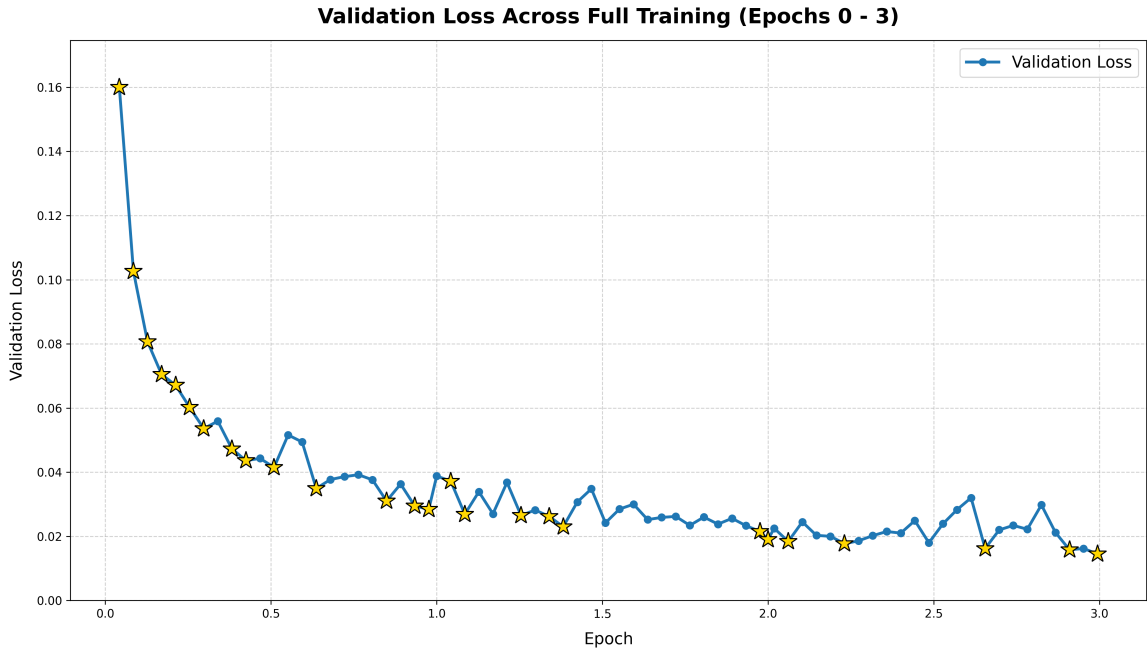


Figure 3: Validation loss across the full 3-epoch training run. The steep initial drop (Epoch 0–0.5) reflects rapid adaptation of pre-trained CodeT5 weights. Gold stars mark “New Best” checkpoints.

The best checkpoint (lowest validation loss) was selected for all downstream evaluations.

4.4 Evaluation Pipeline

For evaluation, we use:

- **BM25 retrieval** (whitespace tokenization) to fetch similar buggy-fixed pairs from the full 52,364 training knowledge base.

- **CodeT5 compressor** to compress demonstrations at the specified τ_{code} .
- **CodeLlama-13B-Instruct** (4-bit quantized) as the base LM for code generation.
- **CodeBLEU** as the evaluation metric, computed across four components: N-gram match, Weighted N-gram match, Syntax match (AST), and Dataflow match.
- 100 randomly sampled test examples (seed=42) for each experiment configuration.

5. Results and Discussion

5.1 Experiment 1: Compression Ratio Sweep (1-shot)

We sweep τ_{code} from 0.0 (no compression) to 0.9 (extreme compression), keeping `num_shots` = 1 fixed.

Table 6: Results of compression ratio sweep (1-shot)

τ_{code}	Actual Compr.	Avg Tok.	CodeBLEU (%)	Syntax (%)	Dataflow (%)
0.0	0% (baseline)	160	91.72	96.82	93.45
0.1	−5.3% (expansion)	168	79.70	83.11	80.95
0.2	6.4%	149	71.49	75.73	72.52
0.3	18.0%	131	69.10	71.81	71.32
0.5	41.1%	94	80.36	83.88	83.03
0.7	64.2%	57	78.61	82.12	80.35
0.9	87.3%	20	78.68	82.23	80.19

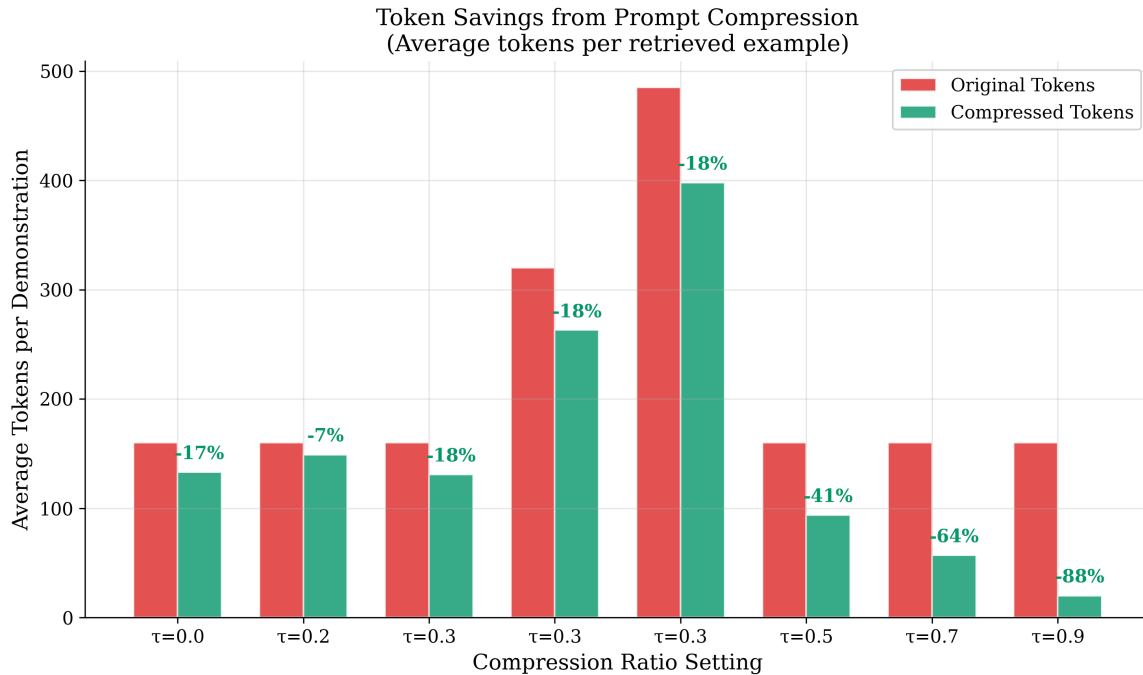


Figure 4: Token savings at each compression ratio. Red bars = original tokens; green bars = compressed tokens. At $\tau = 0.9$, demonstrations shrink from 160 to just 20 tokens (88% saving).

Key findings:

1. **Non-monotonic curve:** The CodeBLEU does not degrade linearly with compression. After an initial dip ($\tau = 0.1$ – 0.3), performance *recovers* at higher compression ($\tau = 0.5$ – 0.9). This suggests that at moderate compression, the compressor removes enough context to confuse the base LM, but at high compression, the demonstrations become so short that the base LM relies more on its internal knowledge, producing reasonable outputs.
2. **$\tau = 0.1$ expansion anomaly:** At $\tau = 0.1$, the compressor actually *increased* the token count (avg 168 vs. original 160). This is because the CodeT5 decoder generates tokens auto-regressively and at very low compression targets, it may reproduce the input with minor additions from the vocabulary distribution.

3. **Optimal trade-off:** $\tau = 0.5$ achieves the best compression-quality balance: 41.1% token reduction with only a 12.4% CodeBLEU drop ($91.72 \rightarrow 80.36$).
4. **Robustness at extreme compression:** Even at $\tau = 0.9$ (87.3% token reduction, avg 20 tokens), CodeBLEU remains at 78.68, only 14.2% below the uncompressed baseline.

5.2 Experiment 2: Multi-Shot Ablation ($\tau_{\text{code}} = 0.3$)

We fix $\tau_{\text{code}} = 0.3$ (the paper’s default) and vary the number of retrieved demonstrations.

Table 7: Multi-shot ablation results at $\tau_{\text{code}} = 0.3$

Shots	Compression	Avg Tokens	CodeBLEU (%)
0 (zero-shot)	N/A	—	91.66
1	18.0%	131	69.10
2	17.9%	263	7.35
3	17.9%	398	4.15

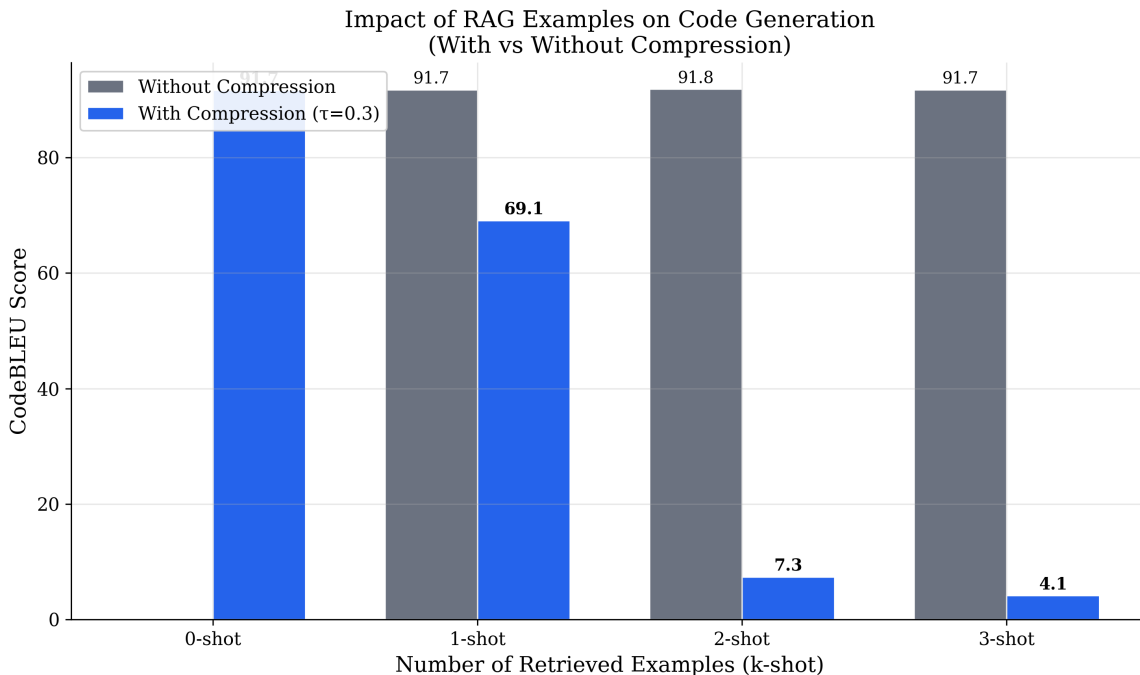


Figure 5: Impact of RAG examples on CodeBLEU, with and without compression ($\tau = 0.3$). Without compression (gray), performance is stable (≈ 91.7). With compression (blue), performance collapses beyond 1-shot.

Key findings:

1. **Zero-shot dominance:** The zero-shot setting achieves 91.66 CodeBLEU, nearly identical to the uncompressed 1-shot baseline (91.72). This indicates that the base LM (CodeLlama-13B) is already highly capable at bug fixing when given just the buggy code.
2. **Multi-shot degradation:** Performance collapses dramatically beyond 1-shot: 2-shot drops to 7.35 and 3-shot to 4.15 CodeBLEU. This is caused by **prompt format saturation**, the CodeLlama model, operating under a fixed 4096 context window with 4-bit quantization, struggles to parse multiple compressed demonstrations.
3. **Implication:** For the Bugs2Fix task with CodeLlama-13B, the optimal configuration is 0-shot or 1-shot with no/light compression.

5.3 Experiment 3: Uncompressed Baselines

To isolate the effect of compression (vs. retrieval itself), we evaluate uncompressed demonstrations at multiple shot counts.

Table 8: Uncompressed baseline results across shot counts

Shots	CodeBLEU (%)	Syntax (%)	Dataflow (%)
1	91.72	96.82	93.45
2	91.82	96.81	93.55
3	91.71	96.90	93.45

Key finding: Without compression, adding more demonstrations has negligible impact (all within $\pm 0.11\%$). This confirms that the base LM is already saturated at 1-shot for Bugs2Fix. This also validates that the multi-shot degradation observed in Experiment 2 is caused by the **compression artifacts**, not by multi-shot prompting itself.

6. How Could We Improve?

6.1 Multi-Method Contextual Bug Fixing

Currently, the pipeline assumes bugs are localized within singular isolated methods. In reality, modern software bugs often span multiple dependent method calls across a larger codebase. We propose extending the framework by including multi-method contexts, where dependent calls are provided alongside the buggy method.

By leveraging static analysis tools (e.g., call graph filters), we can first prune out completely unlinked methods, those outside the function call chain for the buggy logic and then selectively apply our token-pruning compressor on the remaining dependent code blocks. This would effectively upgrade the system into a **multi-method, context-aware prompt compressor** capable of resolving systemic, cross-method bugs.

7. Implementation and Origins

This project is a reproduction and extension of the **CODEPROMPTZIP** framework proposed by Nia et al. (2025). Our implementation was built from scratch based on the methodology described in the original paper and is publicly available.

- **Original Paper:** P. He, S. Wang, and T.-H. Chen, “CODEPROMPTZIP: Code-Specific Prompt Compression for Retrieval-Augmented Generation in Coding Tasks with LMs”
<https://arxiv.org/abs/2502.14925>
- **Our Implementation (GitHub):**
<https://github.com/prakrititz/Token-PruningNLP>
- **Dataset (Microsoft CodeXGLUE – Bugs2Fix):**
<https://github.com/microsoft/codexglue>

Summary

We successfully implemented and evaluated the CODEPROMPTZIP framework for the Bugs2Fix task, demonstrating that code-specific prompt compression can achieve significant token reduction (up to 87.3%) while retaining high downstream quality. The priority-driven greedy compression strategy, combined with the copy-enhanced CodeT5 architecture, produces compressed demonstrations that preserve critical repair signals.

Our experiments reveal that the effectiveness of compressed RAG demonstrations is highly dependent on the base LM’s capacity and context window, suggesting that CODEPROMPTZIP’s benefits are most pronounced when paired with capable, large-context language models.